

Recuperatorio Parcial 2 - 2025

Ejercicio 1:

Se desea evaluar el rendimiento de un bucle de código LEGv8 (64-bit) en un procesador con un *pipeline* de 5 etapas y CPI = 1, con un bus de address de 32 bits.

```

LOOP: LDUR X10, [X2, #0]
      ADDI X10, X10, #1
      STUR X10, [X1, #0]
      ADDI X1, X1, #8
      ADDI X2, X2, #8
      SUBI X3, X3, #1
      CBNZ X3, LOOP
    
```

Especificaciones del Sistema de Memoria:

- Memoria principal direccionada de a bytes.
- Miss Penalty: 50 ciclos.
- Tamaño del Bloque: 32 bytes.
- Caché: Unificada (Datos e Instrucciones), pero supondremos que la caché de instrucciones siempre acierta (100% de HIT de IF) para simplificar. Así, nos enfocaremos solo en los *misses* de datos (LDUR y STUR).
- Política de Escritura: Un *miss* de escritura se trata igual que un *miss* de lectura:

Se deben comparar dos configuraciones de caché de datos:

- Caché A: Capacidad de 1 KB (1024 bytes), Correspondencia Directa.
- Caché B: Capacidad de 1 KB (1024 bytes), Asociativa por Conjuntos de 2 Vías (2-Way), política de reemplazo LRU (Se reemplaza el más antiguo).

El procesador ejecuta el bucle 8 veces (X3 = 8). El puntero al Array A (X1) se inicializa en la dirección: 0x1000 y el del Array B (X2) en la: 0x3000.

a) Determine la cantidad de bits de cada campo del formato de dirección de memoria principal en cada caché.

	Tag	Index/Set	Word	Offset Byte
CACHE A	22	5	2	3
CACHE B	23	4	2	3

$$W_p = 64 \text{ bits} = 8 \text{ bytes}; \text{ Bloque} = 32 \text{ bytes};$$

$$w_m = 8 \text{ bits} = 1 \text{ byte}; \quad \begin{aligned} &\hookrightarrow = 4 \text{ wq}; \text{ Bus de address} = 32 \text{ bits} \\ &\hookrightarrow = 32 \text{ wq}; \end{aligned}$$

CACHE A:

$$\# \text{ Líneas} = \frac{\text{capacidad total}}{\text{capacidad de una línea (bloque)}} = \frac{2^{10} \cdot 2^3}{2^5 \cdot 2^3} = 2^5$$

$$\# \text{ Tags} = \frac{\# \text{ bloques mem}}{\# \text{ líneas cache}} = \frac{\frac{\# w_{mem}}{\# w_{mem} \times \text{bloque}}}{\# \text{ líneas cache}} = \frac{\frac{2^{32}}{2^5}}{2^5} = 2^{22}$$

CACHE B:

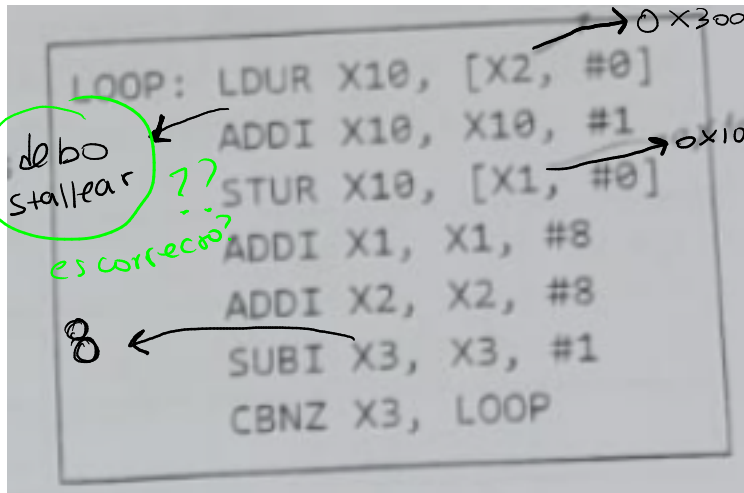
$$\# \text{ Líneas} (\times \text{ vía}) = \frac{\text{cap. total}}{\text{cap. bloque}} \div \text{vías} = \frac{2^{10} \cdot 2^3}{2^5 \cdot 2^3} \div 2 = 2^4$$

$$\# \text{ Tags} = \frac{\# \text{ bloques mem}}{\# \text{ líneas cache}} = \frac{\frac{\# w_{mem}}{\# w_{mem} \times \text{bloque}}}{\# \text{ líneas cache}} = \frac{\frac{2^{32}}{2^5}}{2^4} = 2^{23}$$

b) Calcule el número total de ciclos que toma ejecutar el bucle completo (las 8 iteraciones) para cada configuración de caché. Reporte además el número de MISS registrados

Con CACHE A: 876 ciclos, con 16 misses.

Con CACHE B: 276 ciclos, con 4 misses.



1. if. CACHE A
 $0x3000$: Miss, guardo $[0x3000 - 0x301F]$
 $0x1000$: Miss, reemplazo $[0x1000 - 0x101F]$ (linea 0)
 Siempre qiso lo qe guardé antes
 $\Rightarrow 100\%$ miss rate

Guardo los primeros 8 elementos del arreglo B+1 en el arreglo A y hago 16 accesos a memoria $\Rightarrow 16$ misses

Ciclos =

$$\# \text{instrucciones del código} = (7 + \text{stall}) \cdot 8 = 64$$

(loop entero)

$$\text{Con 5 etapas de pipeline} = 5 + (64 - 1) = 68 \text{ ciclos}$$

$$\text{Con } 100\% \text{ de miss rate en 16 accesos y miss penalty de 50 ciclos} = 68 + 16 \cdot 50 = 868 \text{ ciclos}$$

CACHE 2:

$0x3000$ = Miss, set 0, elijo vía 0, guardo $[0x3000 - 0x301F]$

$0x1000$ = Miss, set 0, elijo vía 1, guardo $[0x1000 - 0x101F]$

Tengo Hit para los otros 6 accesos (acceso a los 1. u elementos de ambos arreglos)

$0x3020$ = Miss, set 1, elijo vía 0, guardo $[0x3020 - 0x303F]$

$0x1020$ = Miss, set 1, elijo vía 1, guardo $[0x1020 - 0x103F]$

Hit para los restantes 6 accesos. $\Rightarrow 4$ misses

$$\text{Ciclos} = 68 + 4 \cdot 50 = 268$$

Ejercicio 2

Un compilador (aka "estudiante") debe optimizar un bucle crítico para un procesador LEGv8 con arquitectura static 2-issue. El hardware tiene un pipeline de 5 etapas (IF, ID, EX, MEM, WB) y ejecuta un paquete de dos instrucciones por ciclo, con las siguientes restricciones:

- **Slot 1:** Ejecuta operaciones ALU (ADD, SUB, ADDI, ORR) o Saltos (B, CBNZ).
- **Slot 2:** Ejecuta operaciones de Carga (LDUR) o Almacenamiento (STUR).

El procesador es estático, tiene todos los caminos de forwarding implementados, y NO

genera stalls por hazards de datos. El compilador (es decir, usted) es responsable de insertar NOPs o reordenar el código para evitar conflictos. Aplicando técnicas de mejora de rendimiento estáticas como reordenamiento, loop unrolling, register renaming, etc., conforme los issue-packets a fin de MINIMIZAR los ciclos de clock necesarios para ejecutar el programa dado (usar la tabla, formato: n° de instrucción > instrucción completa).

Issue	Slot 1	Slot 2
1	1> ADDI X5, XZR, #82	NOP
2	6> SUBI X5, X5, #2	2> LDUR X8, [X5, #0]
3	NOP	3> STUR X8, [X5, #96]
4	NOP	4> LDUR X8, [X5, #1]
5	7> CBNZ X5, L1	5> STUR X8, [X5, #97]
6		
7		
8	>	>
9	>	>

Hago loop unrolling

82 82+96
82 81+96
82 82+96

```

1> ADDI X5, XZR, 82
2> L1: LDUR X8, [X5, #0]
3> STUR X8, [X5, #96]
4> SUBI X5, X5, #1
5> CBNZ X5, L1

LDUR X8 [X5, #-1]
STUR X8 [X5, #95]
SUBI X5, X5, #2
CBNZ X5, L1
  
```

6
7

Ejercicio 3:

Considerar que en un procesador que cuenta con un predictor local de dos bits con una memoria BHT (Branch History Table) de 256 posiciones donde se ejecuta el siguiente código en C, compilado como se muestra a la derecha:

$A[2] = \{17, 2\};$

```
i=2;          //X0<-i
while (i>0){
    if (A[i] < 5) { //X1<-A[i]; X2<-5;
        ...
    }
    if (A[i] > 5) { //X1<-A[i]; X2<-5;
        ...
    }
    i--;
}
```

Dirección	
0x10	cbz X0, end nT
...	...
...	cmp X1, X2
0x34	b.ge s_if1 nT
...	...
0x58	s_if1: b.le s_if2 T nT
...	...
...	s_if2: subi X0, X0, #1

a) Mostrar cómo quedan las posiciones de memoria del predictor luego de ejecutar todo el código y cómo sería la predicción en cada caso. Para esto, considerar que inicialmente el predictor inicia en weakly taken:

10

Dirección	Contenido		Predicción (T/NT)
0001 0000	01	$i = 2$	T
0011 0100	01	$A[2] < 5$	T
0101 1000	11	$A[2] > 5$	T
0001 0000	00	$i = 1$	NT
0011 0100	10	$A[1] < 5$	NT
0101 1000	10	$A[1] > 5$	T
0001 0000	01	$i = 0$	NT

Ejercicio 4:

Suponga un sistema computacional que trabaja a 2 GHz, con hit time = 1 ciclo de clock, CPI = 1 (ideal) y que posee una organización de memoria de dos niveles de jerarquía, una caché y la memoria principal. Asumiendo que en este sistema se ejecuta un programa con la siguiente distribución de instrucciones:

R-Type	CBZ/CBNZ	LDUR	STUR
50%	20%	20%	10%

Las estadísticas de rendimiento son las siguientes:

- Miss Rate de la Caché de Instrucciones (L1-I): 4%
- Miss Rate de la Caché de Datos (L1-D): 12%
- El predictor de saltos del procesador tiene una precisión del 90%.
- La penalización por una predicción de salto errónea es de 4 ciclos de clock.
- La penalidad por un miss de caché es de 100 ns.

Calcular:

- a) Inicialmente, considerar que la caché de instrucciones es perfecta (siempre acierta) ¿cuál es el tiempo promedio de acceso a memoria?

Resultado: 12,44 ns

- b) Suponiendo ahora que la caché de instrucciones tampoco es ideal, ¿cuál es el CPI promedio considerando los fallos de ambas cachés?

CPI_{cache} = 16,2

- c) ¿cuántos CPI extras se producen debido a los fallos de predicción (considerando que la caché es perfecta) usando este método?

$$a) AMAT = \text{Hit time} + \text{miss rate} \times \text{penalty}$$

$$1^\circ \quad 2 \text{ GHz} = \frac{1}{2 \times 10^{-9}} = 0,5 \text{ ns}$$

$$\Rightarrow AMAT = 0,5 \text{ ns} + 0,12 \cdot 100 \text{ ns}$$

$$AMAT = 12,5 \text{ ns}$$

$$b) \quad 100 \text{ ns} = 200 \text{ ciclos}$$

$$\begin{aligned} CPI_{-avg} &= CPI + \text{Memory accesses} \cdot \text{miss rate} \cdot \text{penalty} \\ &= 1 + 0,3 \cdot 0,12 \cdot 200 + 0,04 \cdot 200 \end{aligned}$$

$$CPI_{-avg} = 16,2$$

$$CPI_{\text{predicador}} = 0,08$$

d) El CPI promedio considerando fallos de ambas cache y del predictor de salto:

$$CPI_{\text{Prom}} = 16,28 \quad [b) + c)]$$

e) El equipo de diseño propone dos posibles mejoras para una nueva versión del procesador. Solo se puede implementar una, ¿Cuál de las dos opciones ofrece una mayor mejora en el rendimiento?

- Opción A: Implementar un pre-buscador de instrucciones (prefetcher) que reduce el miss rate de la caché de instrucciones (L1-I) del 4% al 1%.

- Opción B: Mejorar el predictor de saltos, aumentando su precisión del 90% al 98%.

$$CPI_{\text{total}} (\text{Opción A}) = 10,28$$

$$CPI_{\text{total}} (\text{Opción B}) = 16,216$$

$$\text{Mejor Opción} = A$$

$$c) CPI_{\text{pred}} = 0,2 \cdot 0,1 \cdot 4 = 0,08$$

$$e) CPI_{\text{avg}}(A) = 1 + 0,2 \cdot 0,1 \cdot 4 + 0,3 \cdot 0,12 \cdot 200 + 0,01 \cdot 200 = 10,28$$

$$CPI_{\text{avg}}(B) = 1 + 0,2 \cdot 0,02 \cdot 4 + 0,3 \cdot 0,12 \cdot 200 + 0,04 \cdot 200 = 16,216$$

Ejercicio 5:

Un procesador O3 (Out of order) con dynamic multiple issue implementado con el algoritmo de tomasulo se encuentra en el estado que se muestra a continuación, además, se acaba de hacer fetch de las siguientes cuatro instrucciones y esperando en el instruction queue:

SUB X0, X1, X2

CBZ X0, L

ADD X0, X1, X2

Mult X0, X2, X5

Mostrar como quedan las RS y los registros luego de un clock.

Name	Reservation stations						
	Busy	Op	Vj	Vk	Qj	Qk	A
load 1	☒	load	[X2]	-	0	0	[X2] + #0
load 2	☐	load	[X2]	-	0	0	[X2] + #8
store 1	☐	store	-	[X2]	mult fp 1	0	#0
store 2	☐						
alu int 1	☐	addi	[X2]	16	0	0	
alu int 2	☐						
mult int 1	☐						
mult int 2	☐						
alu fp 1	☐	add	-	-	load 1	load 2	
alu fp 2	☐						
mult fp 1	☐	mult	-	[D3]	alu fp 1	0	
mult fp 2	☐						
Branch	☐						

Register Status								
	D0	D1	D2	D3	D4	D5	D6	D7
Q1	alu fp 1	load 1	load 2		mult fp 1			
	X0	X1	X2	X3	X4	X5	X6	X7
Q1			alu int 1					

Name	Reservation stations						
	Busy	Op	Vj	Vk	Qj	Qk	A
Load - Store 1	☒	load	[x2]	-	0	0	x2 + 0
Load - Store 2	☐	load	[x2]	-	0	0	x2 + 8
Load - Store 3	☐	store		[x2]	mult fp 1	0	#0
alu int 1	☒	addi	[x2]	16	0	0	-
alu int 2	☐	SUB	[x1]	-	0	alu int 1	-
alu int 3	☐						-
alu int 4	☐						
alu fp 1	☐	add	-	-	load 1	load 2	-
alu fp 2	☐						
alu fp 3	☐						
alu fp 4	☐						
mult fp 1	☐	Mult	-	[D3]	alu fp 1	0	
mult fp 2	☐						
Branch	☐	CBZ	-	?	alu int 2	0	-

Register Status								
	D0	D1	D2	D3	D4	D5	D6	D7
Q1	alu fp 1	load 1	load 2		mult fp 1			
	X0	X1	X2	X3	X4	X5	X6	X7
Q1	alu int 2		alu int 1					